

PyTorch DDP Deep Dive

How DistributedDataParallel actually works, and how to launch it.

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India

SESSION 2 OF 4

DDP at a Glance

- 1 **One process per GPU** — Not threads — true process isolation
- 2 **Model replicated on every rank** — Each rank holds a full copy of weights + optimizer state.
- 3 **Gradient hook inside backward()** — DDP registers a hook that triggers all-reduce on each grad bucket as soon as it's computed.
- 4 **Comm overlaps with compute** — By the time backward() finishes, most gradients are already reduced — comm is nearly free for large models.
- 5 **Weights stay identical across ranks** — Every rank sees the same averaged gradient and takes the same optimizer step. Weights never drift.

THE BIG IDEA: you write a single-GPU training loop, wrap the model in DDP, launch with torchrun. Done.

Anatomy of a DDP Script (1/2) — Setup

```
import os, torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

def ddp_setup():
    dist.init_process_group(backend="nccl")           # 1. NCCL for GPU
    local_rank = int(os.environ["LOCAL_RANK"])       # 2. bind THIS process
    torch.cuda.set_device(local_rank)
    return local_rank

local_rank = ddp_setup()
device = torch.device(f"cuda:{local_rank}")

model = MyModel().to(device)                        # 3. build model
model = torch.nn.SyncBatchNorm.convert_sync_batchnorm(model) # convert BN
model = DDP(model, device_ids=[local_rank])         # 4. wrap: installs hook
```

- INIT is blocking — every rank waits until all ranks call it, guaranteeing rendezvous
- SET_DEVICE before anything touches GPU, or PyTorch defaults to cuda:0 and ranks collide
- SYNCBN: per-GPU batch under 16 gives noisy BatchNorm stats — convert before wrapping in DDP

Anatomy of a DDP Script (2/2) — Data + Loop

```

world, rank = dist.get_world_size(), dist.get_rank()
sampler = DistributedSampler(dataset, num_replicas=world, rank=rank,      # 5. each rank
                             shuffle=True, drop_last=True)
                             # sees a shard
loader = DataLoader(dataset, batch_size=per_gpu_batch, sampler=sampler,
                    num_workers=4, pin_memory=True)
optim = torch.optim.AdamW(model.parameters(), lr=scaled_lr)

for epoch in range(epochs):
    sampler.set_epoch(epoch)                # 6. CRITICAL: new shuffle order each epoch
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        optim.zero_grad(set_to_none=True)
        loss = model(x, y)
        loss.backward()                    # <-- all-reduce happens HERE
        optim.step()

if rank == 0:    # 7. save from rank 0 only; unwrap .module
    torch.save(model.module.state_dict(), "ckpt.pt")
dist.destroy_process_group()

```

The loop LOOKS identical to single-GPU ; Five things changed: sampler, set_epoch, drop_last, backward() triggers comm, save from rank 0 + unwrap .module.

Launching with torchrun

```
# SINGLE NODE, 2 GPUs
torchrun --standalone --nproc_per_node=2
train_ddp.py
# --standalone runs an in-process c10d
rendezvous.
# --nproc_per_node sets WORLD_SIZE for
this machine.
```

```
# MULTI NODE - set NODE_RANK differently
per node
torchrun --nnodes=2 --nproc_per_node=8 \
  --node_rank=${NODE_RANK} --rdzv_id=my-
job-42 \
  --rdzv_backend=c10d --
rdzv_endpoint=node0:29500 \
  --max_restarts=3 train_ddp.py
```

Env var	Meaning
RANK	Global rank of this process
LOCAL_RANK	Rank within this node (picks your GPU)
WORLD_SIZE	Total processes across all nodes
MASTER_ADDR / PORT	Hostname/port of node 0
NCCL_DEBUG	(optional) INFO for verbose collective logs

DDP Memory Footprint — What Each Rank Holds

DDP trades memory for simplicity: every rank holds a FULL copy of everything. This is exactly what FSDP) shards away — worth seeing the cost explicitly.

Component	Per-rank memory (7B model, AdamW, fp32 master)
Model weights (bf16)	14 GB
Gradients (bf16)	14 GB
Optimizer state (fp32 m + v)	56 GB
Activations (batch-dependent)	varies, often 10-40 GB
TOTAL per GPU, replicated N times	~84-124 GB x N ranks

This is why DDP is the default only while the model fits comfortably: the optimizer state alone (4x the weight size for Adam) dominates, and it's fully duplicated on every single GPU.

Five DDP Issues - You WILL Hit

1 **Forgot `sampler.set_epoch(epoch)`**

Symptom: identical shuffle every epoch — silent convergence bug. Fix: call `set_epoch` at every epoch boundary.

3 **BatchNorm with small per-GPU batch**

Symptom: per-GPU batch ≤ 16 gives noisy BN stats, ~4% worse val accuracy. Fix: `convert_sync_batchnorm` before DDP wrap.

5 **`find_unused_parameters=True` as a crutch**

Symptom: silences a real bug, slows training 10-20%. Fix: find the actual unused parameter; don't hide it.

2 **Saved `.state_dict()` not `.module.state_dict()`**

Symptom: checkpoint has 'module.' prefix on every key, fails to load standalone. Fix: always unwrap `.module` when saving.

4 **Uneven final batch -> hang**

Symptom: one rank finishes early, others wait forever on all-reduce. Fix: `drop_last=True` in `DistributedSampler`.

2-GPU DDP Training Run

- 1 `nvidia-smi topo -m -- topology check: NVLink or PCIe?`
- 2 `./launch.sh --epochs 2 --per-gpu-batch-size 128 -- launch 2-GPU DDP training`
- 3 `NCCL_DEBUG=INFO torchrun --standalone --nproc_per_node=2 train_ddp.py --epochs 1 2>&1 | head -80`
- 4 `./launch.sh --epochs 2 --seed 42 -- run once, note rank 0's val_acc`
- 5 `./launch.sh --epochs 2 --seed 42 -- run again, confirm val_acc matches (deterministic)`
- 6 Confirm: two processes in `ps -ef`, each on a different GPU
- 7 Confirm: `NCCL_DEBUG` picks transport NVL (NVLink), SHM, or IB
- 8 Confirm: `nvidia-smi` shows 90%+ util on both GPUs during backward

Convert a Single-GPU Script to DDP

- 1 Take `train_single.py` (provided)
- 2 Add `ddp_setup(): init_process_group, set_device, return local_rank`
- 3 Wrap the model: `model = DDP(model, device_ids=[local_rank])`
- 4 Replace `shuffle=True` with a `DistributedSampler(dataset, num_replicas=world, rank=rank)`
- 5 Add `sampler.set_epoch(epoch)` inside the epoch loop (see Gotcha #1)
- 6 Guard the checkpoint save with `if rank == 0`, and `unwrap .module.state_dict()`
- 7 Run on 2 GPUs: `torchrun --standalone --nproc_per_node=2 train_ddp.py`
- 8 Report `samples/sec` and `final val_acc` -- compare to a single-GPU run at 2x the batch size

Session 2 Summary

- DDP: one process per GPU, full model replica per rank, gradient hook fires inside backward()
- Comm overlaps with compute via bucketed all-reduce — nearly free for large enough models
- Five script changes vs single-GPU: sampler, set_epoch, drop_last, backward() triggers comm, rank-0 save
- torchrun sets RANK / LOCAL_RANK / WORLD_SIZE / MASTER_ADDR for you — standalone for single-node, rdzv for multi-node
- DDP fully duplicates weights + gradients + optimizer state on every rank — the memory cost FSDP later shards away
- Five gotchas cost real debugging days: set_epoch, .module unwrap, SyncBN, drop_last, find_unused_parameters